Atividade Prática - Aula 05

Upload de Arquivos e Edição de Perfil (Trilhas API + Frontend)

Baseado em: roteiro da **Aula 05 — trilha API (Shortz-App API)** e roteiro da **Aula 05 — trilha Frontend (Shortz-App)**

Pré-requisito: Atividades das Aulas 01 a 04 concluídas nas duas trilhas — `GET /profile/me` funcionando (Aula 04), tela de Meu Perfil ainda como placeholder de teste, guarda de rotas e store de autenticação já funcionando.

Ao final desta atividade, você deve ter, no seu próprio projeto:

Do lado do back-end (API):

- `PUT /api/profile/me` aceitando `multipart/form-data` (nome, bio e, opcionalmente, uma foto), validando os campos, salvando a foto em disco (e apagando a antiga, se não for a padrão), e devolvendo os dados atualizados;
- Uma pasta de uploads servida publicamente via `express.static`;
- Uma constante de validação **corrigida** (`BIO_MAX`, específica para o campo `bio`, em vez de reaproveitar uma constante pensada para outro campo).

Do lado do front-end:

- Um **link novo na Navbar** (criada na Aula 04) para a tela de perfil — sem ele, a tela de hoje seria inalcançável só de clique;
- Um ajuste específico numa única chamada do Axios, para lidar com upload de arquivo sem quebrar o padrão `application/json` usado em todo o resto da aplicação;
- A tela de **Meu Perfil** construída de verdade: dados reais carregados, edição de nome/bio com validação, preview de imagem antes do envio, upload funcional.

Você também deve ser capaz de **explicar com suas palavras**:

- O que é `multipart/form-data`, e por que um arquivo não cabe, na prática, dentro de um corpo JSON;
- Por que a instância do Axios (`api.js`) **não muda** nesta aula, mesmo upload de arquivo exigindo um `Content-Type` diferente do padrão configurado nela;
- Por que validar um campo contra uma constante "de nome parecido, mas pensada para outra coisa" é um erro sutil — e como perceber isso antes de acontecer.

PARTE A — Backend: Upload de Arquivos com Multer

Checklist das tarefas:

- [ ] Pasta `public/uploads/profiles/` criada, com um `default-profile.png` dentro
- [ ] `multer` instalado
- [ ] `BIO_MAX: xxx` adicionado a `config/constants.js`
- [ ] `middlewares/profileMulter.js` criado
- [ ] `require('path')` e a linha de `express.static` adicionados, antes das rotas da API
- [ ] `profileUpdateValidator`, `updateUserProfile`, `updateProfile` e a rota `PUT /profile/me` criados, na ordem correta

Checklist dos testes:

- [ ] Atualização sem foto funciona, mantendo a foto atual
- [ ] Atualização com foto nova funciona, e a foto antiga (se não era a padrão) é removida do disco
- [ ] `GET /uploads/profiles/<qualquer-arquivo>` responde `200`
- [ ] Bio acima de xxx caracteres é recusada com `400`

PARTE B — Frontend: Formulário Multipart e a Tela de Meu Perfil

Checklist desta etapa:

- [ ] Link para a tela de perfil visível na Navbar, só quando logado
- [ ] `.env` atualizado, `utils/media.js` criado
- [ ] `updateProfile(formData)` adicionada — a única chamada de toda a aplicação a sobrescrever o `Content-Type` padrão

Checklist dos testes:

Exclusivamente, navegando a partir da LandingPage, e sem digitar nenhuma URL manualmente, verifique se:

- [ ] Tela carrega os dados reais do usuário ao montar
- [ ] Selecionar uma foto atualiza a prévia instantaneamente, sem chamada de rede
- [ ] Salvar funciona, com e sem trocar de foto

Entrega

Envie em resposta a esta tarefa:

1. Na subpasta `01 - Docs` do seu projeto frontend, dentro de uma nova subpasta `atividade05`:
   - Um print da tela de Meu Perfil carregada com dados reais, em `.jpg`, com o nome `perfil-carregado.jpg`;
   - Um print da prévia de imagem aparecendo antes do envio (com a aba Network mostrando nenhuma chamada nova), em `.jpg`, com o nome `preview-local.jpg`;
   - Um print da aba Network mostrando o `Content-Type: multipart/form-data; boundary=...` da requisição de salvamento, em `.jpg`, com o nome `upload-multipart.jpg`;
   - Um print da Navbar mostrando o link "Meu Perfil" visível quando logado, em `.jpg`, com o nome `navbar-link-perfil.jpg`;
   - Um print de cada `curl` da Etapa 7 (Parte A), em `.jpg`;
   - Atualize `checklists.md` com todos os checklists desta atividade marcados.
2. Um arquivo `.zip` do seu projeto de back-end (sem `node_modules`), nomeado `<NomeSobrenome>-<NomeProjeto>-backend.zip`, agora incluindo `middlewares/profileMulter.js` e a pasta `public/uploads/` (pode remover fotos de teste, mas mantenha o `default-profile.png`);
3. Um arquivo `.zip` do seu projeto de front-end (sem `node_modules`), nomeado `<NomeSobrenome>-<NomeProjeto>-frontend.zip`, agora incluindo `utils/media.js` e `views/profile/MyProfileView.vue` funcional.